CLASE

Estructura y organización de archivos



Organizar el código

Requisitos del proyecto:

- [ ] Paquetes base
- [ ] Usuarios
- [ ] Servidor web
- [ ] Base de datos
- [ ] Firewall
- [ ] Monitoreo
- [ ] Backups

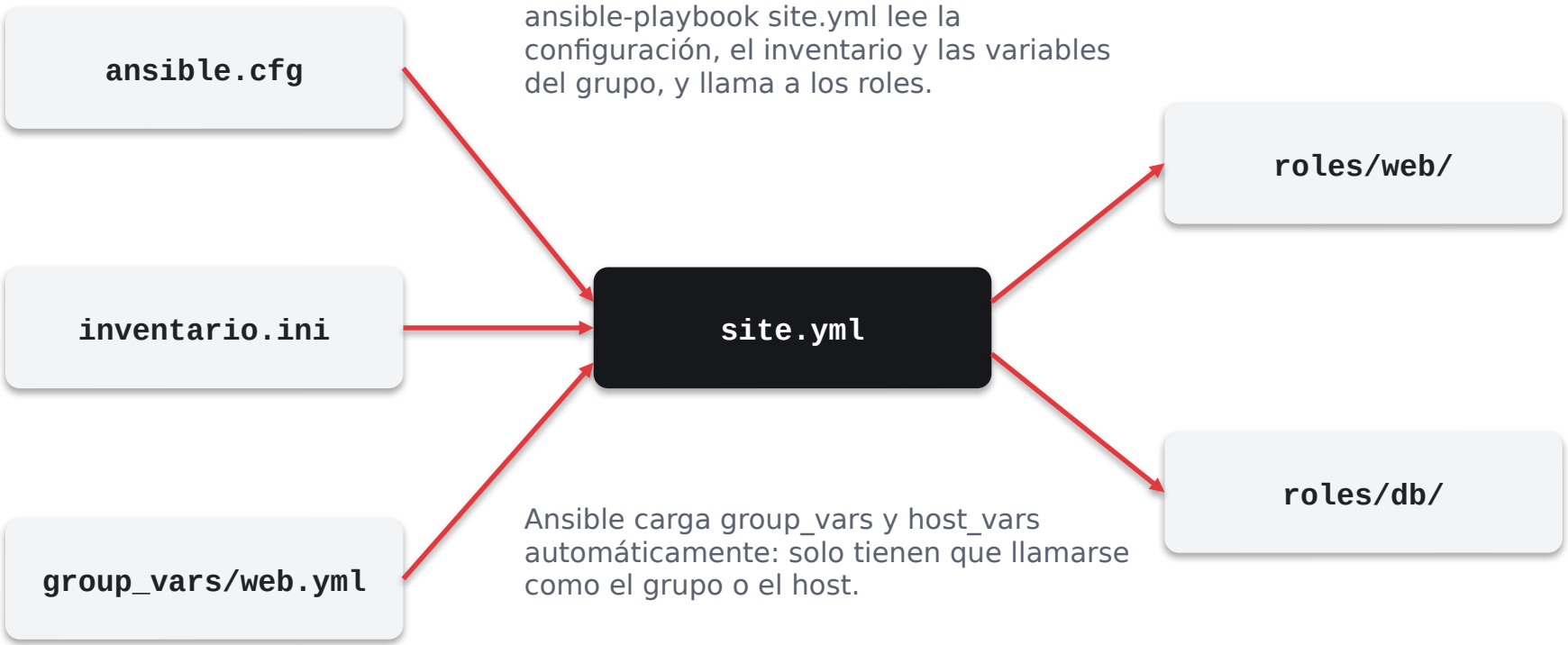




Dividir en archivos



Cómo se conectan los archivos



Archivos comunes

ansible.cfg

Configuración del proyecto: inventario, usuario, clave.

inventario.ini

Hosts y grupos (o un directorio inventory/).

site.yml

Playbook principal que une todo.

group_vars/ host_vars/

Variables por grupo y por host.

roles/

Tareas, plantillas y handlers reutilizables.

requirements.yml

Roles y colecciones a instalar con ansible-galaxy.

árbol

```
proyecto/
├── ansible.cfg
├── inventario.ini
├── site.yml
├── requirements.yml
├── group_vars/
│   └── web.yml
├── host_vars/
│   └── web1.yml
└── roles/
    └── web/
```

ansible.cfg: dónde lo busca Ansible

1. **`ANSIBLE_CONFIG`**
   Variable de entorno

2. **`./ansible.cfg`**
   Directorio actual

3. **`~/.ansible.cfg`**
   Tu carpeta personal

4. **`/etc/ansible/ansible.cfg`**
   Configuración global

Usa el primero que encuentra. Consulta: ansible --version

ansible.cfg

```
[defaults]
inventory = ./inventario.ini
remote_user = ec2-user
private_key_file = ~/.ssh/id_rsa
host_key_checking = False
interpreter_python = auto_silent

[privilege_escalation]
become_method = sudo
```

Variables: group_vars y host_vars

group_vars/web.yml

```yaml
curso: Automatización
paquetes:
  - git
  - tmux
```

host_vars/web1.yml

```yaml
# solo para el host web1
http_port: 8080
```

¿Quién gana? (simplificado)

más prioridad



Roles: código reutilizable

árbol

```
roles/
└── web/
    ├── tasks/main.yml       # tareas
    ├── handlers/main.yml    # handlers
    ├── templates/           # plantillas .j2
    ├── files/               # archivos fijos
    ├── vars/main.yml        # variables
    ├── defaults/main.yml    # valores por defecto
    └── meta/main.yml        # autor, dependencias
```

¿Por qué roles?

- Reutilizar en muchos proyectos
- Compartir en Ansible Galaxy
- Cada cosa en su carpeta
- Playbooks cortos y legibles

TERMINAL

```
$ ansible-galaxy role
    init roles/web
```

Usar roles en un playbook

site.yml

```yaml
---
- name: Servidores web
  hosts: web
  become: true
  roles:
    - web

- name: Bases de datos
  hosts: db
  become: true
  roles:
    - db
```

- El playbook solo dice qué rol aplicar a qué grupo.
- Ansible busca roles/web/tasks/main.yml y lo ejecuta.
- Los handlers, plantillas y variables del rol se cargan solos.
- Roles de la comunidad: ansible-galaxy install -r requirements.yml

Archivos que no van a Git

Claves y .pem

Acceso a los servidores.

.vault_pass

Contraseña de ansible-vault.

inventario.ini

Si contiene IPs reales o se genera solo.

***.retry .ansible/**

Temporales de Ansible y ansible-lint.

.gitignore

```
# Secretos
.vault_pass*
*.pem
*.key
id_rsa*

# Generados
inventario.ini
*.retry
.ansible/
```

Varios entornos, un solo código



Mismos roles y playbooks para todos los entornos. Solo cambia el inventario.

inventories/dev/

hosts.ini + group_vars/

inventories/test/

hosts.ini + group_vars/

inventories/prod/

hosts.ini + group_vars/

TERMINAL

```
$ ansible-playbook \
  -i inventories/dev \
  site.yml

$ ansible-playbook \
  -i inventories/prod \
  site.yml
```

Separar entornos reduce el impacto de un error: pruebas en dev antes de tocar prod.

RESUMEN



Divide un playbook gigante en archivos y carpetas con un propósito claro.



ansible.cfg define la configuración; Ansible lo busca en un orden fijo.



group_vars y host_vars se cargan solos según el inventario.



Los roles agrupan tareas, handlers y plantillas reutilizables.



Claves, contraseñas del vault y temporales no van a Git.



Un inventario por entorno (dev, test, prod) con el mismo código.